

# Practica repaso MD

## Pasaje de mensajes

### Ejercicio 1

1. En una oficina existen 100 empleados que envían documentos para imprimir en 5 impresoras compartidas. Los pedidos de impresión son procesados por orden de llegada y se asignan a la primera impresora que se encuentre libre:
  - a) Implemente un programa que permita resolver el problema anterior usando PMA.
  - b) Resuelva el mismo problema anterior pero ahora usando PMS.

PMA

```
chan envioDocumentos(Documento);

process Empleado[id = 0 to 99] {
    Documento doc;
    doc = GenerarDocumento();
    send envioDocumentos(doc);
}

process Impresora[id = 0 to 4] {
    Documento doc;
    while true {
        receive EnvioDocumentos(doc)
        Imprimir(doc);
    }
}
```

PMS

```
process Empleado[id = 0 to 99] {
    Documento doc;
    doc = GenerarDocumento();
    Coordinador!EnviarDocumento(doc);
}
```

```

}

process Coordinador {
    Documento doc;
    Cola cola;
    do (Empleado[*]?EnviarDocumento(doc)) → {
        cola.push(doc);
    }
    [] (!cola.isEmpty()); Impresora[*]?Atencion(idl) → {
        doc = cola.pop()
        Impresora[idl]!Enviar(doc);
    }
}

process Impresora[id = 0 to 4] {
    Documento doc;
    while true {
        Coordinador!Atencion(id);
        Coordinador?Enviar(doc);
        Imprimir(doc);
    }
}

```

## Ejercicio 2

Resolver el siguiente problema con PMS. En la estación de trenes hay una terminal de SUBE que debe ser usada por P personas de acuerdo con el orden de llegada. Cuando la persona accede a la terminal, la usa y luego se retira para dejar al siguiente. Nota: cada Persona usa sólo una vez la terminal.

```

process Persona[id = 0 to P-1] {
    Coordinador!SolicitaTerminal(id);
    Coordinador?UsarTerminal();
    UsarTerminal()
}

```

```

    Coordinador!LiberarTerminal();
}

process Coordinador {
    boolean libre = true;
    int id;
    Cola cola;
    do (!libre); Persona[*]?SolicitarTerminal(id) → {
        cola.push(id);
    [] libre; Persona[*]?SolicitarTerminal(id) → {
        libre = false;
        Persona[id]!UsarTerminal()
    }
    [] Persona[*]?LiberarTerminal()
        if (cola.isEmpty())
            libre = true;
        else
            id = cola.pop(cola);
            Persona[id]!UsarTerminal()
    }
}

```

## Ejercicio 3

```

chan Solicitud(Integer, TipoPersona, Integer)
chan Boleta(Integer, TipoPersona, Integer)
chan Recibo[P](Integer, Integer)
chan CajeroLibre()

chan Turno[P]();

process Persona[id = 0 to P-1] {
    Integer id, cantBoletas, dinero, vuelto;
    TipoPersona tipoPersona;
    Boleta boletas;
    Recibo recibo;
}

```

```

    send Solicitud(id, tipoPersona, cantBoletas);
    receive Turno[id]()

    send Boleta(id, boletas, dinero);
    receive Recibo[id](recibo, vuelto);
}

process Coordinador {
    Cola cola; // Asumimos que la cola agrega por prioridad de embarazadas
    (tipoPersona) y cantBoletas.
    do Receive Solicitud(id, tipoPersona, cantBoletas) → {
        cola.push(id, tipoPersona, cantBoletas);
    }
    [] (!cola.isEmpty()) Receive CajeroLibre() → {
        id = cola.pop();
        send Turno[id]()
    }
}

process Cajero {
    while true {
        Send CajeroLibre();
        Receive Boleta(id, boletas, dinero);

        // Me devuelve por parametro de salida valores a vuelto y recibo
        Trabajar(boletas, dinero, vuelto, recibo);

        Send Recibo[id](recibo, vuelto)
    }
}

```

Version con un canal por prioridad

```

chan SolicitudAnciano(Integer, Integer, Boleta);
chan SolicitudMenosDe5(Integer, Integer, Boleta);
chan SolicitudNormal(Integer, Integer, Boleta);
chan Recibo[P](Integer, Integer)

```

```

process Persona[id = 0 to P-1] {
  Integer id, cantBoletas, dinero, vuelto;
  TipoPersona tipoPersona;
  Boleta boletas;
  Recibo recibo;

  if (tipoPersona == "Anciano")
    send SolicitudAnciano(id, dinero, boletas);
  else if (cantBoletas <= 5)
    send SolicitudMenosDe5(id, dinero,boletas);
  else
    send SolicitudNormal(id, dinero, boletas);

  receive Recibo[id](recibo, vuelto);
}

process Cajero {
  while true {
    if (not empty(SolicitudAnciano))
      receive SolicitudAnciano(id, dinero, boletas);
    else if (not empty(SolicitudMenosDe5))
      receive SolicitudMenosDe5(id, dinero, boletas);
    else
      receive SolicitudNormal(id, dinero, boletas);

    // Trabaja
    Send Recibo[id](recibo, vuelto)
  }
}

```

## ADA

### Ejercicio 1

Resolver el siguiente problema. La página web del Banco Central exhibe las diferentes cotizaciones del dólar oficial de 20 bancos del país, tanto para la compra como para la

venta. Existe una tarea programada que se ocupa de actualizar la página en forma periódica y para ello consulta la cotización de cada uno de los 20 bancos. Cada banco dispone de una API, cuya única función es procesar las solicitudes de aplicaciones externas. La tarea programada consulta de a una API por vez, esperando a lo sumo 5 segundos por su respuesta. Si pasado ese tiempo no respondió, entonces se mostrará vacía la información de ese banco.

```
Procedure Banco_Central is
```

```
Task Pagina is
```

```
end Pagina;
```

```
Task type Banco is
```

```
    entry ValorDolar(valorDolarCompra: in float, valorDolarVenta: in float);  
end Banco;
```

```
vectorBancos: array[1..20] of Banco;
```

```
task body Pagina is
```

```
    compra, venta: float;
```

```
    vectorValoresCompra: array[1..20] of float;
```

```
    vectorValoresVenta: array[1..20] of float;
```

```
    begin
```

```
        for i in 1..20 loop
```

```
            SELECT
```

```
                Banco[i].ValorDolar(compra, venta) do
```

```
            OR
```

```
                delay(5) do
```

```
                    compra = null;
```

```
                    venta = null;
```

```
                vectorValoresCompra[i] = compra;
```

```
                vectorValoresVenta[i] = venta;
```

```
            end loop;
```

```
    end Pagina;
```

```
task body Banco is
```

```

    compra, venta: float;
begin
    loop
        accept ValorDolar(compra: out float, venta: out float) do
            getValorDolar(compra, venta);
        end ValorDolar;
    end loop;
end Banco;

begin
end Banco_Central;

```

## Ejercicio 2

Resolver el siguiente problema. En un negocio de cobros digitales hay  $P$  personas que deben pasar por la única caja de cobros para realizar el pago de sus boletas. Las personas son atendidas de acuerdo con el orden de llegada, teniendo prioridad aquellos que deben pagar menos de 5 boletas de los que pagan más. Adicionalmente, las personas ancianas tienen prioridad sobre los dos casos anteriores. Las personas entregan sus boletas al cajero y el dinero de pago; el cajero les devuelve el vuelto y los recibos de pago.

```

procedure Cobros is

    task type Persona is
        entry recibirl(id: in int);
        entry turno();
    end Persona;
    vectorPersonas: array[1..P] of Persona;

    task Cajero is
        entry atender(boletas: in string, dinero: in int, recibo: out string, vuelto: out int);
    end Cajero;

    task Coordinador is

```

```

    entry solicitarTurno(id: in int, tipoPersona: in Tipo, cantBoletas: in int)
end Coordinador;

task body Persona is
begin
    accept recibirId(id: in int);
    Coordinador.solicitarTurno(id, tipoPersona, cantBoletas);
    accept turno();
    Cajero.atender(boletas, dinero, recibo, vuelto);
end Persona;

task body Cajero is
    while true loop
        Coordinador.cajeroLibre();
        accept atender(boletas: in string, dinero: in int, recibo: out string, vuelto: out int) do
            Trabajar(boletas, dinero, recibo, vuelto);
        end atender;
    end loop;
end Cajero;

task body Coordinador is
begin
    while true loop
        SELECT
            accept solicitarTurno(id: in int, tipoPersona: in Tipo, cantBoletas: in int) do
                cola.push(id, tipoPersona, cantBoletas);
            end solicitarTurno;
        OR
            when (not cola.isEmpty()); accept cajeroLibre() do
                id = cola.pop();
                Persona[id].turno();
            end cajeroLibre();
    end loop;
end Coordinador;

begin

```



```

    for i in 1..P loop
        Persona[i].recibirId(i);
    end loop;
end

```

Version con un canal para cada prioridad

```

procedure Cobros is

    task type Persona;
    vectorPersonas: array[1..P] of Persona;

    task Cajero is
        entry atenderAnciano(boletas: in string, dinero: in int, recibo: out string, v
        uelto: out int)
        entry atenderMenosDe5(boletas: in string, dinero: in int, recibo: out strin
        g, vuelto: out int)
        entry atenderNormal(boletas: in string, dinero: in int, recibo: out string, v
        uelto: out int)
    end Cajero;

    task body Persona is
    begin
        cantBoletas = getCantBoletas();
        Tipo = getTipoPersona();
        if (Tipo == "Anciano")
            Cajero.atenderAnciano(boletas, dinero, recibo, vuelto);
        else if (cantBoletas <= 5)
            Cajero.atenderMenosDe5(boletas, dinero, recibo, vuelto);
        else
            Cajero.atenderNormal(boletas, dinero, recibo, vuelto);
        end Persona;

    task body Cajero is
        loop
            SELECT
                accept atenderAnciano(boletas: in string, dinero: in int, recibo: out st
                ring, vuelto: out int) do

```

```

        Trabajar(boletas, dinero, recibo, vuelto);
    end atenderAnciano;
OR
    when (atenderAnciano'count == 0); accept atenderMenosDe5(bolet
as: in string, dinero: in int, recibo: out string, vuelto: out int) do
        Trabajar(boletas, dinero, recibo, vuelto);
    end AtenderMenosDe5;
OR
    when (atenderAnciano'count == 0) and (atenderMenosDe5'count =
= 0); accept atenderNormal(boletas: in string, dinero: in int, recibo: out strin
g, vuelto: out int) do
        Trabajar(boletas, dinero, recibo, vuelto);
    end atenderNormal;
end loop;
end Cajero;

begin
end Cobros;

```

## Ejercicio 3

Resolver el siguiente problema. La oficina central de una empresa de venta de indumentaria debe calcular cuántas veces fue vendido cada uno de los artículos de su catálogo. La empresa se compone de 100 sucursales y cada una de ellas maneja su propia base de datos de ventas. La oficina central cuenta con una herramienta que funciona de la siguiente manera: ante la consulta realizada para un artículo determinado, la herramienta envía el identificador del artículo a las sucursales, para que cada una calcule cuántas veces fue vendido en ella. Al final del procesamiento, la herramienta debe conocer cuántas veces fue vendido en total, considerando todas las sucursales. Cuando ha terminado de procesar un artículo comienza con el siguiente (suponga que la herramienta tiene una función generarArtículo() que retorna el siguiente ID a consultar). Nota: maximizar la concurrencia. Existe una

función ObtenerVentas(ID) que retorna la cantidad de veces que fue vendido el artículo con identificador ID en la base de la sucursal que la llama.

```
procedure Empresa is

task Central is
    entry RespuestaConsulta(ventas: in int);
end Central;

task type Sucursal is
    entry ConsultaArticulo(articulo: in int);
end Sucursal;
vectorSucursales: array[1..100] of Sucursal;

task body Central is
    art: integer;
    ventasArticulo: integer;
begin
    while true loop
        ventasArticulo = 0;
        art = generarArticulo();
        for i in 1..100 loop
            Sucursal[i].ConsultaArticulo(art);
        end loop;
        for i in 1..100 loop
            accept RespuestaConsulta(cantVentas: in int) do
                ventasArticulo += cantVentas;
            end loop;
        end loop;
    end loop;
end Central;

task body Sucursal is
    ventas: integer;
begin
    while true loop
        accept ConsultaArticulo(articulo: int int)
        ventas = obtenerVentas(articulo);
        Central.RepuestaConsulta(ventas);
    end loop;
end Sucursal;
```

```
    end loop;  
end Sucursal;  
  
begin  
end Empresa;
```

Segunda version, quiza mas concurrente

```
procedure Empresa is  
  
    task Central is  
        entry RespuestaConsulta(ventas: in int);  
    end Central;  
  
    task type Sucursal is  
        entry ConsultaArticulo(articulo: in int);  
    end Sucursal;  
    vectorSucursales: array[1..100] of Sucursal;  
  
    task body Central is  
        art: integer;  
        ventasArticulo: integer;  
        cola: Cola;  
    begin  
        art = generarArticulo();  
        while (art <> -1) loop  
            ventasArticulo = 0;  
  
            for i in 1..100 loop  
                Sucursal[i].Atencion();  
            end loop;  
  
            for i in 1..200 loop  
                SELECT  
                    accept ConsultaArticulo(articulo: out int) do  
                        articulo = art;
```

```

        end ConsultaArticulo;
    OR
        accept RespuestaConsulta(cantVentas: in int) do
            ventasArticulo += cantVentas;
        end RespuestaConsulta;
    end loop
    cola.push(art, ventasArticulo);
    art = generarArticulo();
end loop;
end Central;

task body Sucursal is
    ventas, articulo: integer;
begin
    while true loop
        accept Atencion();
        Central.ConsultaArticulo(articulo)
        ventas = obtenerVentas(articulo);
        Central.RepuestaConsulta(ventas);
    end loop;
end Sucursal;

begin
end Empresa;

```